



Instituto Politécnico de Coimbra
Instituto Superior de Engenharia de Coimbra

Trabalho Prático

Deep Sea Mining

Unidade Curricular: Programação Avançada
ANO LETIVO 2025/2026

Diogo Pinto – 2024152576

Vera Ribeiro – 2024140140

Rafael Marques – 2024143044

Índice

1	Introdução	2
2	Explicação do Jogo	3
3	Opções Tomadas	4
3.1	Minijogo - <i>Sliding Puzzle</i>	4
3.2	FossoState	4
3.3	DeepSeaCanvas	4
3.4	BarraLateralBase	4
4	Diagrama de Padrões	5
4.1	Finite State Machine	5
4.2	Factory	6
4.3	Observer	6
4.4	Singleton	6
4.5	Facade	7
5	Diagrama de Estados	8
6	Diagrama de Classes	9
7	Descrição das Classes	10
7.1	Aplicação	10
7.2	Modelo de Dados – Elementos	10
7.3	Modelo de Dados – Grelhas	11
7.4	Modelo de Dados – Jogo e Puzzle	11
7.5	Máquina de Estados (FSM)	12
7.6	<i>Facade</i> e Utilitários	12
7.7	Interface Gráfica (UI)	13
8	Opções Tomadas na UI	14
8.1	Superfície (Navegação do navio)	14
8.2	Oficina (Gestão de recursos)	14
8.3	Fosso (Descida e Subida)	14
8.4	Fundo	15
8.5	Puzzle	15
8.6	Fim do Jogo	15
9	Resumo das Tarefas Previstas	16
9.1	1ª meta – Modelo de dados	16
9.2	2ª meta – Gestão do Jogo e Persistência de Dados	17
9.3	3ª meta – Interface Gráfica JavaFX	18
9.4	Fase Final – Sons e documentação	19
10	Conclusão e considerações finais	19

1 Introdução

O presente relatório foi realizado no âmbito da Unidade Curricular de Programação Avançada e tem como objetivo descrever o desenvolvimento de um jogo de estratégia e exploração "Deep Sea Mining".

A aplicação desenvolvida permite realizar um jogo de estratégia e exploração subaquática, suportando todas as regras fundamentais do jogo, incluindo a navegação do navio na superfície, a descida de drones, a recolha de minérios e artefactos no fundo do mar, e a gestão crítica de recursos (combustível e integridade dos drones). Adicionalmente, foram implementadas funcionalidades como o modo de Oficina para gestão de melhorias e reparação dos drones, a resolução de puzzles para obtenção dos artefactos, e uma interface gráfica (UI) com efeitos sonoros, que se adapta ao estado do jogo, de acordo com o solicitado ao longo das metas do projeto.

A estrutura do projeto foi organizada segundo o padrão arquitetural MVC (Model-View-Controller), garantindo uma separação rigorosa entre a lógica de negócio (Modelo) e a interface gráfica (Vista/Controlador). A camada de mediação e o único ponto de acesso ao modelo é feita através da classe DeepSeaManager, que atua como *Facade*. a comunicação do Modelo para a Interface Gráfica é feita de forma assíncrona com recurso ao Padrão Observer (utilizando PropertyChangedSupport), assegurando que o Modelo desconhece por completo a existência da UI.

O projeto foi desenvolvido de forma coerente, acompanhando as etapas propostas ao longo do semestre.

2 Explicação do Jogo

Deep Sea Mining é um jogo de exploração submarina onde o jogador controla um navio e três drones de mineração com o objetivo de recolher artefactos dispersos pelo fundo do mar. O jogador ganha quando todos os artefactos são recolhidos e transportados até ao navio, ou perde se o navio ficar sem combustível ou se todos os drones forem destruídos.

O navio navega numa grelha de superfície e consome um determinado valor de combustível por movimento. Cada posição da superfície corresponde a uma zona submarina distinta, com o seu próprio fosso e fundo.

Para explorar o fundo, o jogador escolhe um dos drones disponíveis e inicia a descida pelo fosso. Durante a descida e a posterior subida, o drone encontra obstáculos gerados aleatoriamente: rochas que formam as paredes do fosso, animais marinhos que causam dano, e correntes que aumentam o consumo de combustível. Os obstáculos mudam de posição a cada expedição, tornando cada viagem diferente.

Ao atingir o fundo, o drone pode explorar células ocultas, o mapa vai sendo revelado à medida que avança. No fundo encontra minérios que são recolhidos com consumo extra de combustível, e artefactos que exigem a conclusão de um minijogo de puzzle para serem recolhidos. Existem também monstros que causam dano ao drone. Qualquer tipo de dano sofrido, seja por rochas, animal marinho ou monstro, cresce a cada impacto segundo uma sequência dos números primos. Esta sequência é reiniciada a cada expedição.

O minijogo é um *sliding puzzle* com um limite de movimentos. Se o jogador ordenar as peças dentro desse limite, o artefacto é recolhido e a exploração continua. Caso contrário, o drone inicia a subida sem ele.

Após efetuar a subida, ao regressar à superfície, o drone descarrega automaticamente tudo o que transporta para o navio.

Na superfície, o jogador pode aceder à Oficina em qualquer momento para:

- Abastecer o tanque do drone;
- Reparar o casco do drone.

Ambas as operações têm custo em combustível do navio. É possível também:

- Melhorar a capacidade do tanque do drone;
- Melhorar a integridade máxima do drone.

Estas melhorias são permanentes e têm custo em minérios.

Se um drone for destruído (integridade ou combustível a zero), perde tudo o que transportava e é removido definitivamente do jogo.

3 Opções Tomadas

3.1 Minijogo - *Sliding Puzzle*

O enunciado define que o minijogo deve gerir 15 peças numa grelha 4×4 com contador de movimentos, deixando a implementação ao critério do grupo. Optámos por um *sliding puzzle*: existe uma célula vazia que troca de posição com uma peça adjacente a cada jogada, e o objetivo é ordenar todas as peças em sequência numérica dentro do limite de movimentos.

Para baralhar o puzzle, em vez de colocar as peças em posições aleatórias, aplicamos um número de movimentos válidos a partir do estado resolvido. Esta abordagem garante que o puzzle é sempre solucionável.

3.2 FossoState

O enunciado pede os estados de descida e subida como classes distintas. Como ambos partilham a mesma lógica de movimentação no fosso, consumo de combustível, colisão com obstáculos e verificação de destruição do drone, criámos uma classe abstrata **FossoState** para centralizar esse código e evitar duplicação. Cada subclasse acrescenta apenas a verificação de chegada ao destino e a transição de estado correspondente.

3.3 DeepSeaCanvas

O enunciado pede um **Canvas** derivado por cada momento do jogo. Criámos então a classe abstrata **DeepSeaCanvas**, que deriva da classe **Canvas** e centraliza o comportamento comum a todos os momentos: guardar referência ao **DeepSeaManager**, calcular o tamanho das células, redimensionar proporcionalmente a grelha quando a janela muda, e declarar os métodos abstratos `update()` e `registerHandlers()`. Com esta classe abstrata, **SuperficieCanvas**, **FossoCanvas**, **FundoCanvas** e **PuzzleCanvas** implementam apenas o que é específico ao seu momento.

3.4 BarraLateralBase

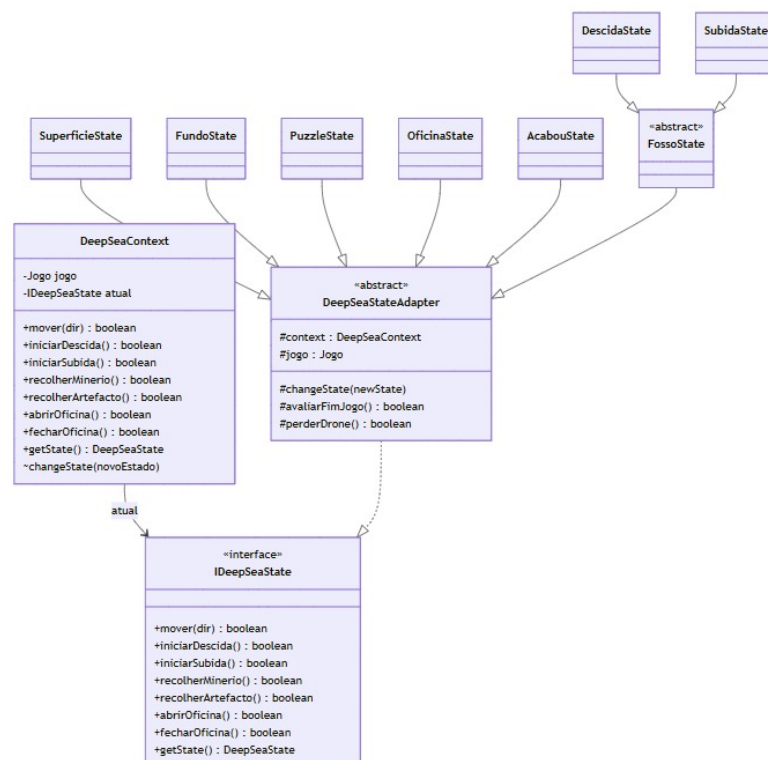
De forma semelhante ao **DeepSeaCanvas**, criámos a classe abstrata **BarraLateralBase** para centralizar o que é comum a todas as barras laterais: estrutura visual, cabeçalho com título, subtítulo e autores, e área reservada para conteúdo específico. Os métodos abstratos `update()` e `registerHandlers()` são implementados nas subclasses. Desta forma, cada barra lateral preenche apenas a área central com a informação relevante ao seu estado.

4 Diagrama de Padrões

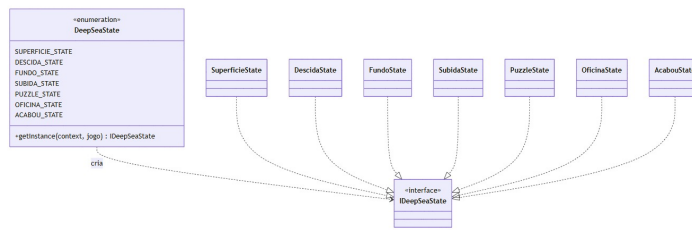
Neste projeto, e de acordo com o pretendido no enunciado, utilizámos os seguintes padrões de *design*:

- **FSM** – Gere os diferentes estados do jogo (superfície, descida, fundo, subida, *puzzle*, oficina e acabou) através de uma máquina de estados, delegando o comportamento de cada ação ao estado atual, através do **DeepSeaContext** e do **IDeepSeaState**.
- **Factory Method** – Cria dinamicamente as instâncias dos estados a partir do *enum* **DeepSeaState**, através do método **getInstance()**, separando a criação dos estados da sua utilização.
- **Observer** – Notifica de forma automática a *UI* sempre que há uma mudança de estado no jogo, colisões com objetos, movimentações (navio, *drone*) e atualização de dados utilizando o **PropertyChangeSupport** no **DeepSeaManager**.
- **Singleton** – Garante que o **DeepSeaLog** possua uma única instância partilhada por toda a aplicação.
- **Facade** – Simplifica e protege a comunicação entre a interface gráfica e a lógica de jogo, centralizando todas as operações possíveis de serem realizadas pelo utilizador no **DeepSeaManager**, que delega internamente para o **DeepSeaContext** e para a classe **Jogo**.

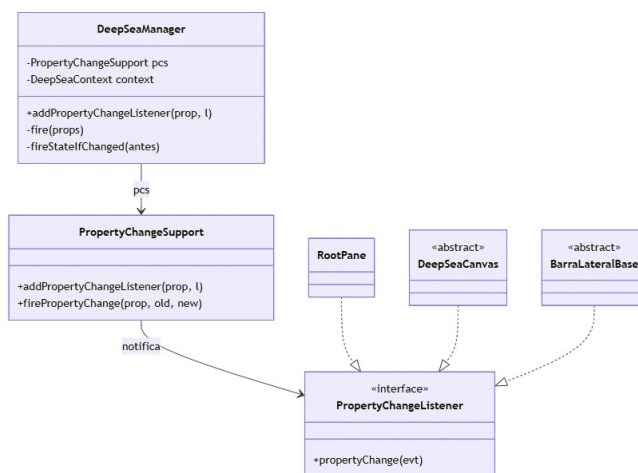
4.1 Finite State Machine



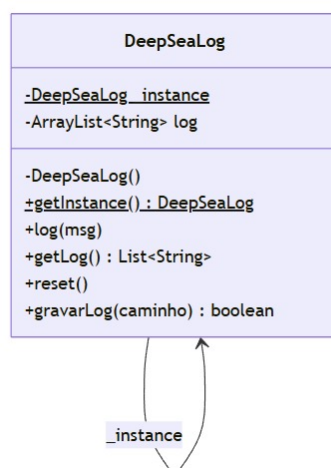
4.2 Factory



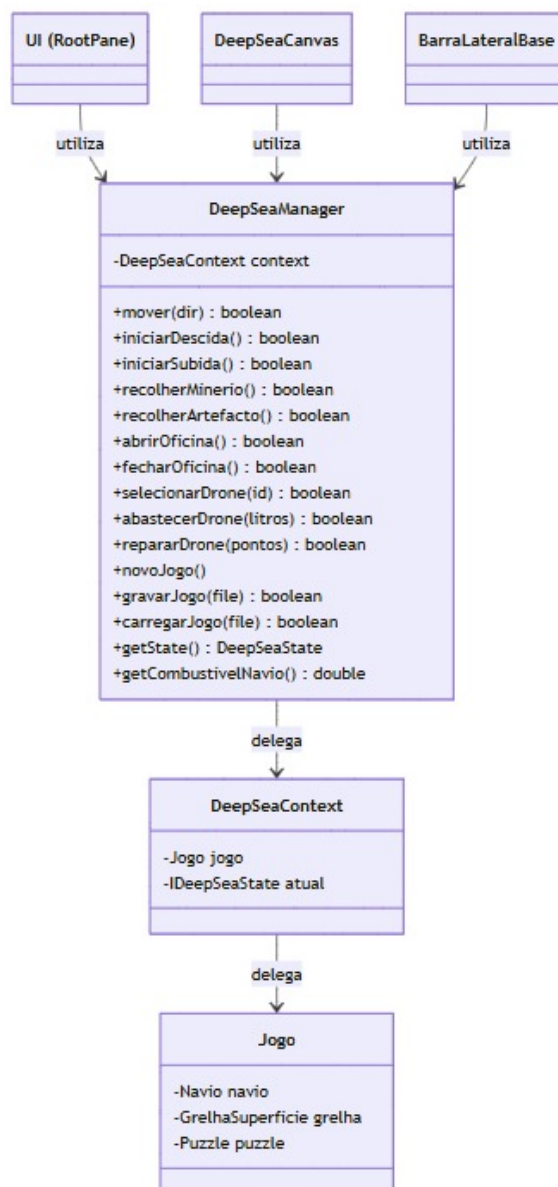
4.3 Observer



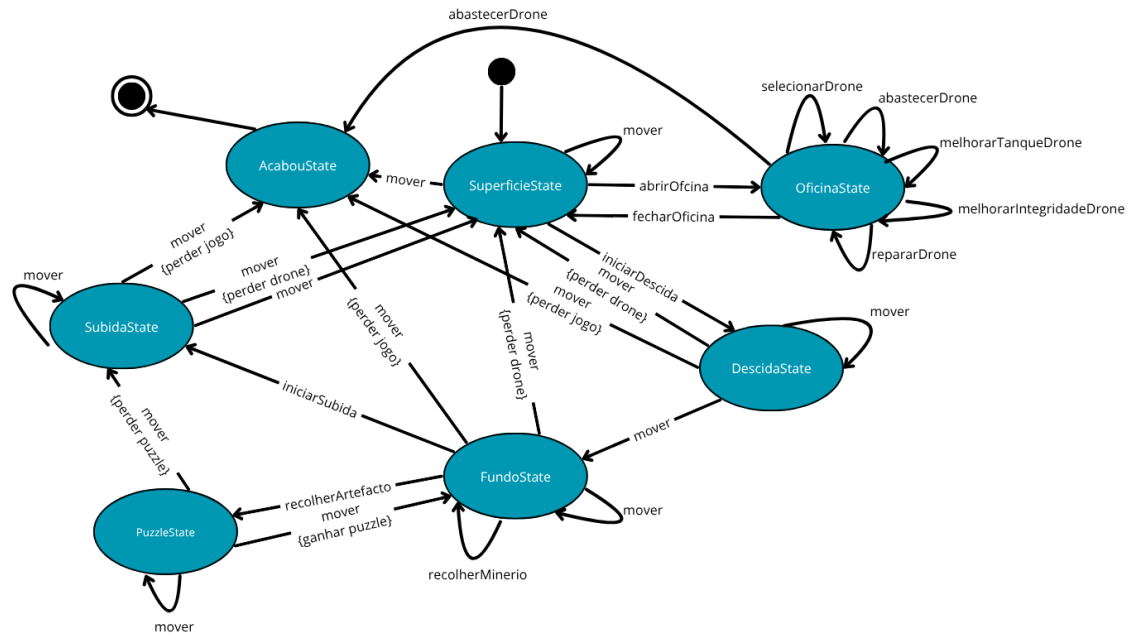
4.4 Singleton



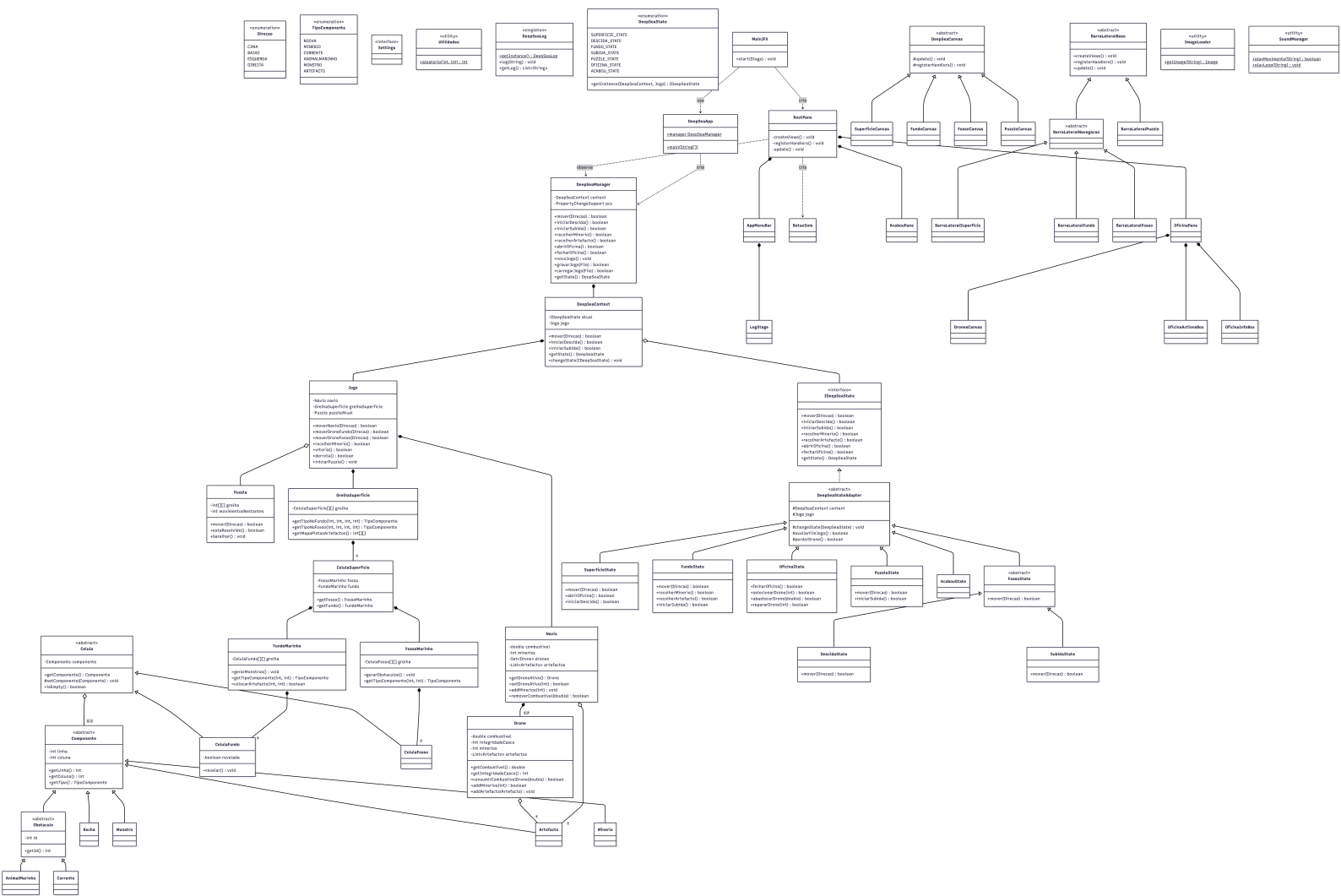
4.5 Facade



5 Diagrama de Estados



6 Diagrama de Classes



7 Descrição das Classes

Nas tabelas seguintes apresenta-se uma breve descrição das principais classes que constituem a aplicação, organizadas de acordo com a arquitetura adotada.

7.1 Aplicação

Classe	Descrição
DeepSeaApp	Ponto de entrada da aplicação; executa a <code>Application.launch(MainJFx.class,args)</code> para inicializar a classe <code>MainJFX</code> .

Tabela 1: Classe de arranque da aplicação

7.2 Modelo de Dados – Elementos

Classe	Descrição
Componente	Classe base dos elementos do jogo que podem existir numa célula.
Obstaculo	Classe abstrata para os obstáculos do fosso, nomeadamente: Rocha, Corrente e AnimalMarinho.
Rocha	Classe derivada de Obstáculo que forma as paredes do fosso que provoca dano ao <i>drone</i> .
AnimalMarinho	Classe derivada de Obstáculo que representa um animal marinho que pode existir numa célula do fosso que provoca dano ao <i>drone</i> .
Corrente	Classe derivada de Obstáculo que representa uma corrente marinha que pode existir numa célula do fosso que provoca o dobro do consumo do combustível ao <i>drone</i> .
Minerio	Classe derivada de Componente que representa um minério que pode existir numa célula do fundo marinho, e que contém quantidades variadas do minério.
Artefacto	Classe derivada de Componente que representa um artefacto que pode existir numa célula do fundo marinho, e cuja recolha necessita da resolução de um puzzle.
Monstro	Classe derivada de Componente que representa um monstro que pode existir numa célula do fundo marinho que provoca dano ao <i>drone</i> .
TipoComponente	Enumeração dos tipos de componente existentes no jogo.

Tabela 2: Modelo de dados – elementos

7.3 Modelo de Dados – Grelhas

Classe	Descrição
GrelhaSuperficie	Grelha bidimensional da superfície onde o navio navega.
FossoMarinho	Grelha do fosso de cada célula da superfície onde o <i>drone</i> navega na descida e subida.
FundoMarinho	Grelha do fundo de cada célula da superfície, com minérios, monstros, artefactos. Contém as suas células escondidas que apenas se revelam quando visitadas pelo <i>drone</i> , mantendo-se reveladas após o mesmo.
Celula	Classe base das células das grelhas do fundo e fosso.
CelulaSuperficie	Classe que representa cada célula da grelha de superfície.
CelulaFosso	Classe derivada da classe base Célula e que representa cada célula da grelha do fosso.
CelulaFundo	Classe derivada da classe base Célula e que representa cada célula da grelha do fundo. Contém a informação do seu estado (revelada ou não revelada).

Tabela 3: Modelo de dados – grelhas

7.4 Modelo de Dados – Jogo e Puzzle

Classe	Descrição
Jogo	Classe principal que contém o Navio e a Grelha da Superfície. Permite aceder aos elementos do modelo de dados.
Navio	Representa o navio (contém combustível, minérios, artefactos, 3 <i>drones</i> e uma localização na grelha da superfície).
Drone	Representa o <i>drone</i> (contém integridade do casco, localização (no fosso, fundo, não tendo localização na superfície por se encontrar dentro do navio, contém ainda recursos transportados como minérios e artefactos).
Puzzle	Classe que representa o minijogo <i>sliding puzzle</i> . Contém a grelha do puzzle e o respetivo contador de movimentos.

Tabela 4: Modelo de dados – jogo e *puzzle*

7.5 Máquina de Estados (FSM)

Classe	Descrição
DeepSeaContext	Contexto da FSM. Controla o estado atual e redireciona os pedidos.
DeepSeaState	Enumeração dos estados do jogo com o respetivo <i>Factory Method</i> .
IDeepSeaState	Interface com a declaração de todas as transições de estado.
DeepSeaStateAdapter	Classe abstrata com a implementação por omissão das transições da qual derivam cada um dos estados.
SuperficieState	Estado que representa a navegação do navio na superfície.
FossoState	Classe abstrata com a lógica comum à descida e subida no fosso da qual derivam os estados <i>DescidaState</i> e <i>SubidaState</i> .
DescidaState	Estado que representa a descida do <i>drone</i> pelo fosso.
SubidaState	Estado que representa a subida do <i>drone</i> pelo fosso.
FundoState	Estado que representa a exploração do <i>drone</i> no fundo marinho.
PuzzleState	Estado que representa a realização do minijogo <i>slidding puzzle</i> que permite a recolha de artefactos.
OficinaState	Estado que representa a oficina que permite realizar a gestão de recursos e melhorias dos <i>drones</i> , bem como seleccionar o drone a usar.
AcabouState	Estado final do jogo (vitória ou derrota).

Tabela 5: Máquina de estados (FSM)

7.6 *Facade* e Utilitários

Classe	Descrição
DeepSeaManager	<i>Facade</i> entre a UI e o modelo de dados, que concentra todas as operações e as notificações (<i>PropertyChangeSupport</i>).
DeepSeaLog	Registo (<i>log</i>) das operações sobre o modelo, seguindo o padrão <i>Singleton</i> .
Settings	Interface que contém as constantes utilizadas em toda a aplicação.
Utilidades	Métodos auxiliares de uso geral.
Direcao	Enumeração das direções de movimento.

Tabela 6: *Facade* e utilitários

7.7 Interface Gráfica (UI)

Classe	Descrição
MainJFX	Classe que arranca o JavaFX (extends Application). Cria o Stage e Scene.
RootPane	Classe que representa o painel raiz que sobrepõe os <i>canvas</i> dos momentos do jogo e as barras laterais nos respectivos StackPane .
AppMenuBar	Classe que representa a barra de menu com as opções: novo jogo, abrir, gravar, <i>logs</i> , som e sair.
BotaoSom	Classe que representa o botão de ligar/desligar o som.
LogStage	Janela de apresentação dos <i>logs</i> .
AcabouPane	Classe que representa o ecrã de fim de jogo (<i>Game Over</i>).
DeepSeaCanvas	<i>Canvas</i> abstrato com o comportamento comum a todos os momentos.
SuperficieCanvas	<i>Canvas</i> que permite representar a interface da superfície.
FossoCanvas	<i>Canvas</i> do fosso que permite representar a interface (descida/subida).
FundoCanvas	<i>Canvas</i> que permite representar a interface do fundo marinho.
PuzzleCanvas	<i>Canvas</i> que permite representar a interface do minijogo.
BarraLateralBase	Classe abstrata com a estrutura comum a todas as barras laterais.
BarraLateralNavegacao	Classe abstrata que estende a classe BarraLateralBase representa os elementos e comportamentos comuns às barras laterais de navegação da superfície, fundo e fosso.
BarraLateralSuperficie	Barra lateral da interface de superfície.
BarraLateralFosso	Barra lateral da interface do fosso.
BarraLateralFundo	Barra lateral da interface do fundo.
BarraLateralPuzzle	Barra lateral do minijogo.
OficinaPane	Classe que permite representar a interface da oficina.
DronesCanvas	<i>Canvas</i> que apresenta os <i>drones</i> para seleção na oficina.
OficinaInfoBox	Caixa de informação da oficina.
OficinaActionsBox	Caixa de ações da oficina (abastecer, reparar, melhorar).
ImageLoader	Classe que permite realizar o carregamento das imagens dos elementos do jogo.
SoundManager	Classe que permite carregar e gerir os sons da aplicação.

Tabela 7: Interface gráfica (UI)

8 Opções Tomadas na UI

A interface gráfica do nosso jogo *Deep Sea Mining*, desenvolvida em JavaFX, tem como foco a simplicidade, usabilidade e a separação visual de responsabilidades.

De forma a garantir que o jogo proporcione uma experiência intuitiva, dividimos o ecrã em duas partes. A zona esquerda (que ocupa a maior parte do ecrã) é destinada à apresentação das grelhas de jogo através de elementos *Canvas*, enquanto na margem direita temos um painel de informação relativo ao jogo, onde são apresentadas as quantidades de combustível e integridade, o *drone* selecionado e, no caso de estarmos no *puzzle*, o número de movimentos restantes.

Os elementos dos jogo (obstáculos, monstro, drone, navio e rochas) foram representados com recurso a imagens.

Abaixo, detalhamos as principais opções de *design* tomadas para cada momento do jogo.

8.1 Superfície (Navegação do navio)

A grelha de superfície do nosso jogo foi implementada em tons de azul-claro (LIGHT-BLUE), transmitindo a ideia de água e permitindo realçar a presença do navio.

As posições que possuem cores amarelas, laranjas ou vermelhas indicam a presença de artefactos no fundo do mar nessa exata localização. Cada cor representa a quantidade de artefactos presentes: amarelo indica 1 artefacto, laranja indica 2 artefactos, e vermelho indica a presença de 3 ou mais artefactos no fundo dessa célula marinha.

Na barra lateral, barras de progresso mostram o combustível do navio e do *drone* ativo, bem como a integridade do casco. Para iniciar a descida no fosso e para abrir a oficina, existem na barra lateral dois botões dispostos lado a lado, bem visíveis e de fácil acesso para o utilizador. No topo, destaca-se também a *menu bar*, onde temos opções de acesso rápido, como começar um novo jogo, ver os *logs*, silenciar o som, entre outras funcionalidades.

8.2 Oficina (Gestão de recursos)

A oficina substitui a grelha por um ecrã dedicado à gestão de recursos, onde podemos seleccionar o *drone* a utilizar na próxima descida, abastecer e reparar os *drones* e, se possível, melhorar os seus tanques de combustível e integridade.

Os três *drones* são apresentados lado a lado, cada um no seu próprio *Canvas*, e a seleção atual é evidenciada por um contorno azul que destaca qual o veículo ativo no momento.

8.3 Fosso (Descida e Subida)

Para transmitir uma sensação de profundidade, a paleta de cores foi alterada para um tom de azul mais escuro.

Nas laterais do fosso existem rochas, representadas por blocos texturizados esverdeados. Existem também correntes e animais marinhos: os animais são representados por um polvo, e as correntes por uma imagem de “ondas”, que ilustra visualmente a força da água.

8.4 Fundo

O fundo marinho é representado por um tom de azul ainda mais escuro, contendo artefactos, minérios e monstros marinhos. Inicialmente, caso o jogo não esteja em “modo defesa”, todo o fundo encontra-se coberto (em escuridão), não sendo visível para o utilizador, sendo gradualmente revelado à medida que este o explora com o *drone*. Tomámos esta decisão para cumprir os requisitos do enunciado e para tornar a exploração mais realista e imersiva.

8.5 Puzzle

O *puzzle* é um minijogo de 15 peças organizado numa grelha 4×4 . Os números são apresentados sobre um fundo azul-claro, e a posição vazia não contém qualquer valor. O utilizador move as peças utilizando as setas do teclado. O número de movimentos restantes é visível na barra lateral e, ao chegar a zero, o *drone* inicia a subida automaticamente.

8.6 Fim do Jogo

Quando o jogo termina, é apresentada no ecrã a mensagem “*Game Over*” e a contagem total de artefactos recolhidos durante a partida, juntamente com duas opções: “Jogar Novamente” e “Sair”.

9 Resumo das Tarefas Previstas

Nas tabelas seguintes apresentam-se os requisitos implementados em cada uma das etapas de realização do trabalho.

9.1 1ª meta – Modelo de dados

Requisito / Tarefa	Estado
Elementos necessários à modelação do jogo com recurso à herança e polimorfismo.	Implementado
Classe principal Jogo que agrega todas as componentes do modelo de dados.	Implementado
Grelha da superfície como um array bidimensional único.	Implementado
Grelha que representa o fosso marinho de cada célula da superfície onde estão representados os elementos existentes nas fases de subida e descida (rochas, animais marinhos e correntes marítimas).	Implementado
Grelha que representa o fundo marinho de cada célula da superfície onde estão representados os elementos existentes no fundo do mar (minérios, monstros e artefactos) e a informação que permite verificar se a célula foi ou não revelada.	Implementado
Classe Navio que representa o navio bem como todas as suas propriedades (quantidade de combustível, minérios, artefactos recolhidos, coleção de Drones e localização).	Implementado
Classe Drone que representa o drone bem como todas as suas propriedades (integridade do casco, localização e recursos transportados). Localização inicial do drone na descida e na subida no centro do fosso e no fundo do mar no centro da grelha.	Implementado
Mecanismo de geração da grelha da superfície.	Implementado
Mecanismo de geração de cada uma das grelhas correspondentes aos fossos marinhos de cada uma das células da grelha da superfície. Geração das rochas de cima para baixo, garantindo que a posição da rocha mais interior de cada margem não pode oscilar mais de uma coluna face à linha anterior e que a soma das rochas de ambos os lados em cada linha não ultrapassa 50% da largura da grelha.	Implementado
Mecanismo de geração de cada uma das grelhas correspondentes aos fundos marinhos de cada uma das células da grelha da superfície, incluindo a localização dos artefactos e dos minérios, tendo estes últimos quantidades variadas em cada localização.	Implementado
Métodos que permitem consultar e ordenar os drones por quantidade de combustível ou por integridade do casco.	Implementado
Definição do minijogo com o respetivo contador de movimentos.	Implementado
Interface para especificação das diversas contantes que definem as diversas vertentes do jogo.	Implementado
Modo de defesa que permite facilitar os testes das funcionalidades durante a defesa do trabalho.	Implementado
Testes JUnit que permitem testar as classes principais do modelo de dados.	Implementado

Tabela 8: 1ª meta – Modelo de dados

9.2 2ª meta – Gestão do Jogo e Persistência de Dados

Requisito / Tarefa	Estado
Evolução do jogo gerida recorrendo a uma máquina de estados (FSM – Finite State Machine).	Implementado
Diagrama de estados do jogo que identifica os estados e transições de estados que refletem todas as possibilidades de evolução no jogo, em PDF na pasta reports do projeto.	Implementado
Classe DeepSeaContext implementada no contexto da FSM, agindo como Facade da FSM, controlando o seu estado atual, redirecionando os pedidos para o estado atual e disponibilizando métodos get necessários para aceder aos dados.	Implementado
Enumeração DeepSeaState de todos os estados bem como o respetivo Factory Method.	Implementado
Interface IDeepSeaState com a declaração de todas as transições de estado.	Implementado
Classe abstrata DeepSeaStateAdapter com a implementação por omissão dos métodos das transições e de mecanismos auxiliares para transição de estados, recorrendo ao contexto da FSM, com uma referência para o contexto e para o modelo de dados.	Implementado
Classes com a implementação de todos os estados concretos (SuperficieState, SubidaState, PuzzleState, OficinaState, FundoState, FossoState, DescidaState e AcabouState).	Implementado
Classe DeepSeaManager que permite representar todo o modelo de dados, passando por ela todos os pedidos ao modelo e obtenção de respostas do mesmo.	Implementado
Processos de serialização e desserialização que permitem que todas as informações essenciais do jogo sejam guardadas para que um jogo possa ser salvo-guardado.	Implementado
Classe DeepSeaLog que permite efetuar o log de todas as operações que são realizadas sobre o modelo de dados, seguindo o padrão Singleton.	Implementado
Associação da data/hora em que foi reportada cada mensagem recebida pelo log. Permitir operações de consulta, gravação para ficheiro de texto e eliminação de mensagens. Nome do ficheiro de log definido juntamente com as outras constantes da aplicação.	Implementado
Testes JUnit que permitem testar as classes principais do modelo de dados.	Implementado
Classes principais comentadas com Javadoc.	Implementado

Tabela 9: 2ª meta – Gestão do Jogo e Persistência de Dados

9.3 3ª meta – Interface Gráfica JavaFX

Requisito / Tarefa	Estado
UI que permite gerir todas as etapas do jogo.	Implementado
Funcionalidades de salvaguarda e restauro de um jogo e apresentação e gravação dos logs gerados ao longo da execução.	Implementado
Interação do utilizador através do rato ou de teclas de atalho para a escolha de operações e controlo dos elementos do jogo.	Implementado
Elementos do tipo Canvas para a apresentação das grelhas dos vários momentos do jogo (superfície, fossos, fundos e minijogo), definidos seguindo o modelo do RootPane do MVC@PA, sobrepostos co recurso a um StackPane, estando apenas visível o componente correspondente.	Implementado
Representação dos elementos de cada grelha (navio, drone, rocha, corrente, animal, monstro, minério e artefacto) no ecrã através de imagens armazenadas na pasta images existente em src/main/resources.	Implementado
Caixas laterais para apresentação de informação geral do jogo (combustível do navio, drones, níveis de combustível, nível de integridade do casco, minérios, artefactos recuperados, entre outros).	Implementado
Menu da aplicação que permite realizar as seguintes opções: iniciar um novo jogo (New), restaurar um jogo previamente gravado com recurso ao FileChooser (Open...), apresentar um submenu com os últimos 5 jogos gravados (Open Recent...), gravar o jogo (Save as...), sair do jogo(Exit), mostrar ou esconder a janela dos logs (Show/Hide), gravar os logs com recurso ao FileChooser indicando o nome do ficheiro (Save logs) e limpar todos os logs (Clear logs).	Implementado
Mecanismos necessários para que a UI possa receber notificações relativas a alterações aos dados do modelo e logs, recorrendo a Property Changes.	Implementado
Classes principais documentadas em Javadoc.	Implementado

Tabela 10: 3ª meta – Interface Gráfica JavaFX

9.4 Fase Final – Sons e documentação

Requisito / Tarefa	Estado
Sons armazenados na pasta resources/sounds.	Implementado
Classe SoundManager que permite gerir os sons incorporados na aplicação.	Implementado
Gerar sons em cada movimento do drone ou do navio bem como em momentos especiais como quando o drone se sobrepõe a um animal ou quando é destruído.	Implementado
Gestão de sons permitindo tocar dois sons seguidos de forma correta.	Implementado
Opção de ligar ou desligar os sons da aplicação, gerida através de uma property do JavaFX Beans do tipo boolean na classe SoundManager.	Implementado
Ajustamento da visualização das grelhas, em função proporcional ao tamanho da janela, quando alterada pelo utilizador.	Implementado
Finalização dos testes unitários às classes principais do modelo de dados.	Implementado
Segunda janela em tudo semelhante àquela onde é apresentada a interface com o utilizador, com o conteúdo sempre em concordância com o estado do jogo, e qualquer delas podendo ser usada pelo utilizador para interagir com o jogo.	Implementado
Finalização da documentação Javadoc das classes principais.	Implementado
Realização do relatório final.	Implementado

Tabela 11: Etapa Final – Sons, entrega e apresentação final

10 Conclusão e considerações finais

O desenvolvimento deste jogo permitiu consolidar conhecimentos fundamentais sobre Java, interfaces com JavaFX e, principalmente a aplicação de padrões de design de software em projetos reais.

A implementação do jogo *Deep Sea Mining* foi bem sucedida e resultou numa aplicação funcional, estável e com uma base sólida de design, refletindo boas práticas de programação orientada a objetos.